

VIRAL-CORE — Claude Code Super Prompt

Viral Property Sharing, Referral Hierarchy, Personalization & Network Growth Suite

Extension to the existing Real Estate Platform (Branches 8-15)

0. MISSION

You are extending an **existing real estate platform** (Next.js 14 App Router · TypeScript · Supabase · monorepo) with a new module suite codenamed **VIRAL-CORE**. The mind map below (Branches 8-15) is the source of truth for *what* to build. Your job is to turn it into production code following the conventions in this prompt.

One-line product definition: every registered user becomes a promoter; every asset they share (brochure, flyer, property photo, QR, link, social post) automatically carries their identity + tracking, and anyone who registers through that asset is mapped into the sharer's hierarchy under configurable rules — producing a self-growing sales, referral, recruitment and commission network.

Build VIRAL-CORE as a set of shared engine packages plus app surfaces. Do **not** rewrite or break Branches 1-7 (existing listings, search, marketplace, auth). Integrate with them.

1. ASSUMPTIONS (stated, not asked — correct me only if wrong)

1. The platform already has: a Supabase Postgres DB, Supabase Auth, a `users`/`profiles` table, RBAC roles, property listings, and Supabase Storage buckets.
2. Money math uses **Decimal.js** in the app layer and `numeric` in Postgres. **No IEEE-754 floats for money**. Audit logs are append-only/immutable.
3. The existing design language is institutional/luxury (dark canvas, crimson/gold accents per the real-estate brand). VIRAL-CORE surfaces inherit the existing design tokens — do not invent a new theme.
4. **KaroChat** is a first-party share channel (treat it as a native integration target alongside WhatsApp/Telegram, not a third-party OAuth).
5. AI text generation routes through the **Claude API** (existing NEXUS pattern). Image enhancement routes through a **provider-agnostic adapter** (configurable).

If any assumption is wrong, surface it in your first reply and propose the corrected approach before generating code.

2. NON-NEGOTIABLE CONVENTIONS

- **Stack:** Next.js 14 (App Router, RSC where sensible), TypeScript strict, Supabase (Postgres + Auth + Storage + Realtime + Edge Functions), Tailwind, shared packages in `/packages/*`.
 - **Money:** Decimal.js everywhere; `numeric(20,8)` columns; never `float`/`number` for currency. Format INR with the existing rupee utility.
 - **IDs:** UUID v4 PKs. Referral codes are short, human-shareable, collision-checked (base32, 8 chars).
 - **Auditability:** every commission, attribution decision, reward grant and lead status change writes an immutable row. No hard deletes on ledgers — use status flags + reversal rows.
 - **RLS:** Row Level Security ON for every new table. Users see their own subtree; admins/state-heads see scoped trees; service role for engines.
 - **Validation:** Zod schemas at every boundary (API, server action, Edge Function).
 - **No secrets in client.** Compositing, AI calls, attribution resolution, and commission math run server-side only (Route Handlers / Edge Functions / server actions).
 - **Idempotency:** click logging, conversion attribution, and commission distribution must be idempotent (dedupe keys), because they fire from unreliable network events.
-

3. PACKAGE & APP ARCHITECTURE

Create these shared packages (or merge into existing equivalents if the monorepo already has them):

```
packages/
├─ referral-engine/          # codes, QR, hierarchy (closure table),
attribution, commission distribution
├─ personalization-engine/  # template slots, server-side image/PDF
compositing, QR overlay, share-link minting
├─ content-capture/        # photo→ad generator, EXIF/GPS, "Captured Live"
badge compositor
├─ field-discovery/        # property-lead capture, moderation queue,
publish-to-marketplace
├─ gamification-engine/    # points, achievements, leaderboards, rewards
ledger
├─ ai-content/             # Claude-API copy generation + image-enhancement
adapter
├─ viral-shared/           # zod schemas, types, share-channel registry,
constants
```

App surfaces (extend existing apps; do not fork new apps unless one doesn't exist):

- **Referral Dashboard** (every user): my tree, my links/QR, my shares, my commissions, leaderboard.
- **Brochure / Asset Studio** (every user): pick asset → auto-personalize → share to channel.
- **Capture & Ad Studio** (every user): take/upload photo → auto-generated ad in N formats → share.
- **Field Discovery** (agent role): submit property leads.
- **Admin Console** (admin): media library, templates, attribution/commission config, moderation queue, fraud review, rewards.

4. DATA MODEL (Postgres / Supabase migrations)

Generate these as ordered SQL migrations. `numeric(20,8)` for money. All tables: `id uuid` pk default `gen_random_uuid()`, `created_at timestamptz default now()`, RLS enabled.

4.1 Identity & referral spine (extends existing user/profile)

```
sql
```

```
-- Extend existing profile (do NOT create a second users table)
alter table profiles
  add column referral_code      text unique,          -- base32, 8 chars
  add column share_slug         text unique,          -- /r/<slug>
  add column qr_storage_path    text,
  add column referrer_id        uuid references profiles(id), -- direct sponsor
  add column promoter_status    text default 'active',
  add column profile_photo_path text;

-- Closure table = fast arbitrary-depth ancestor/subtree queries + per-level co
create table referral_closure (
  ancestor_id  uuid not null references profiles(id),
  descendant_id uuid not null references profiles(id),
  depth        int  not null,          -- 0 = self
  primary key (ancestor_id, descendant_id)
);
create index on referral_closure(descendant_id, depth);
create index on referral_closure(ancestor_id, depth);
```

Design choice: closure table is primary (clean N-levels-up commission queries + unlimited depth). Adjacency edge (`referrer_id`) is kept for simple parent lookups. Postgres `ltree` is an acceptable alternative if you'd rather store a materialized path — pick one and be consistent; default to closure table.

4.2 Tracking & attribution

sql

```

create table referral_clicks (
  id uuid primary key default gen_random_uuid(),
  referral_code text not null,
  promoter_id uuid references profiles(id),
  asset_id uuid, -- personalized_renders.id or capture
  channel text not null, -- whatsapp|telegram|karoachat|facebook
  landing_slug text,
  utm jsonb default '{} '::jsonb,
  ip_hash text, -- hashed, not raw
  device_fp text, -- fingerprint
  user_agent text,
  clicked_at timestamptz default now(),
  converted_user uuid references profiles(id),
  dedupe_key text unique -- idempotency
);

create table attribution_config (
  id uuid primary key default gen_random_uuid(),
  model text not null default 'first_click', -- first_click | last_click
  window_days int not null default 30,
  cookie_days int not null default 30,
  allow_override boolean default false, -- last-click override toggle
  updated_by uuid references profiles(id),
  updated_at timestamptz default now()
);

```

4.3 Commission / revenue ledger (immutable)

sql

```
create table commission_plan (  
  id uuid primary key default gen_random_uuid(),  
  name      text not null,  
  max_levels int not null,          -- depth cap for payout  
  level_rates jsonb not null,      -- { "1":0.40, "2":0.10, ... } fractions  
  basis      text not null default 'sale_value', -- sale_value | platform_fee  
  active      boolean default true  
);  
  
create table revenue_ledger (  
  id uuid primary key default gen_random_uuid(),  
  sale_id      uuid not null,      -- FK to existing sales/transactions  
  beneficiary_id uuid not null references profiles(id),  
  source_user_id uuid not null references profiles(id),  
  level         int not null,  
  basis_amount  numeric(20,8) not null,  
  rate          numeric(10,8) not null,  
  amount        numeric(20,8) not null,  
  plan_id       uuid references commission_plan(id),  
  status        text not null default 'accrued', -- accrued|approved|paid|reversed  
  reversal_of   uuid references revenue_ledger(id),  
  created_at    timestampz default now()  
);  
-- No updates that change amount; corrections = new reversal row.
```

4.4 Media library & personalization

sql

```

create table media_assets (
  id uuid primary key default gen_random_uuid(),
  type      text not null,    -- brochure|flyer|video|image|template|campaign|
  title      text not null,
  storage_path text not null,
  thumb_path text,
  status      text default 'published',
  uploaded_by uuid references profiles(id)
);

create table asset_overlay_templates (
  id uuid primary key default gen_random_uuid(),
  asset_id uuid references media_assets(id),
  -- slots: array of {key, type:'text'|'image'|'qr', x,y,w,h, font, size, color}
  -- binding ∈ user_name|user_mobile|user_photo|referral_code|qr|company_logo|w
  slots      jsonb not null
);

create table personalized_renders (
  id uuid primary key default gen_random_uuid(),
  user_id     uuid references profiles(id),
  asset_id    uuid references media_assets(id),
  channel      text,
  output_path text,
  share_link   text,
  qr_path      text,
  created_at  timestampz default now()
);

```

4.5 Content capture → auto-ad

sql

```

create table property_captures (
  id uuid primary key default gen_random_uuid(),
  user_id    uuid references profiles(id),
  photo_path text not null,
  gps        geography(point),          -- enable postgis
  captured_at timestampz,
  device_meta jsonb,                   -- exif subset
  category   text, price numeric(20,8), area_text text, location_text text,
  verified   boolean default false
);

create table generated_ads (
  id uuid primary key default gen_random_uuid(),
  capture_id uuid references property_captures(id),
  format      text not null,           -- social_post|flyer|story|status|reel_
  output_path text,
  created_at  timestampz default now()
);

```

4.6 Field discovery

```

sql

create table property_leads (
  id uuid primary key default gen_random_uuid(),
  agent_id    uuid references profiles(id),
  gps         geography(point),
  media_paths text[] default '{}',
  voice_note_path text,
  owner_details_enc bytea,             -- encrypt PII at rest
  price       numeric(20,8),
  description text,
  status      text default 'submitted', -- submitted|verifying|approved|publish
  reviewer_id uuid references profiles(id),
  listing_id  uuid,                   -- set when published to marketplace
  created_at  timestampz default now()
);

```

4.7 Gamification

```

sql

```



```

create table achievements (
  id uuid primary key default gen_random_uuid(),
  key text unique, title text, description text, icon text, points int default 0
);
create table user_achievements (
  id uuid primary key default gen_random_uuid(),
  user_id uuid references profiles(id), achievement_id uuid references achievements(id),
  earned_at timestamptz default now(), unique(user_id, achievement_id)
);
create table points_ledger (
  id uuid primary key default gen_random_uuid(),
  user_id uuid references profiles(id), delta int not null, reason text,
  ref_type text, ref_id uuid, created_at timestamptz default now()
);
create table rewards (
  id uuid primary key default gen_random_uuid(),
  type text, title text, value numeric(20,8), criteria jsonb
);
create table reward_grants (
  id uuid primary key default gen_random_uuid(),
  user_id uuid references profiles(id), reward_id uuid references rewards(id),
  status text default 'granted', granted_at timestamptz default now()
);
-- Leaderboards: materialized views refreshed on a schedule (referrals, sales,

```

5. MODULE BUILD SPECS (mapped to Branches 8-15)

Module A — Registration & Referral Spine (Branch 8)

- Email **or** mobile registration → OTP verify → identity validation → profile creation.
- On successful verify: generate `referral_code`, `share_slug`, QR (PNG to Storage), personal sharing link `/r/<slug>`.
- **Automatic hierarchy assignment:** if registration carried an attributed referrer (see Module C), insert the closure rows (copy ancestor set of referrer + self-row at depth 0). This is the atomic "join the tree" operation — wrap in a transaction.
- Post-registration: unlock access to brochures, marketing materials, videos, documents, images, social creatives, landing pages, referral dashboard.

Module B — Personalization Engine (Branch 9)

- Admin uploads assets + defines `slots` per template (visual slot editor: drag boxes onto the asset, bind each to a data field).
- **Render pipeline (server-side):** load template → resolve bindings for the requesting user (name, mobile, photo, referral code, QR, company logo/watermark, contact/WhatsApp buttons, trackable link) → composite.
 - Images: `sharp` (Node) in an Edge Function or Route Handler.
 - PDFs/brochures: `pdf-lib` overlay, or render an HTML template → PDF.
 - Always burn in the QR + a unique `share_link` that encodes `referral_code` + `asset_id` + `channel`.
- Output saved to `personalized_renders`, returned with per-channel deep links.
- **Share channels:** WhatsApp, Telegram, **KaroChat (native)**, Facebook, Instagram, X, LinkedIn, Email, SMS, Copy Link. Each channel = a registry entry with a link-builder + (where supported) prefilled text.

Module C — Attribution & Auto-Hierarchy (Branch 10)

- `/r/<slug>` and QR scans hit a Route Handler that: logs a `referral_clicks` row (idempotent via `dedupe_key`), sets a first-party attribution cookie (configurable days), then redirects to the personalized landing page.
- On registration completion (OTP verified), resolve attribution by `attribution_config.model`:
 - **first_click** (default): earliest click in window for this device/cookie.
 - **last_click**: most recent (only if `allow_override`).
- Write `converted_user` back to the click row and run the closure-insert transaction (Module A).
- **Commission distribution** (on confirmed sale, not on signup): walk closure ancestors (where depth between 1 and `plan.max_levels`), compute each level's `amount = basis_amount * level_rate` (Decimal.js), insert immutable `revenue_ledger` rows (`accrued`). Idempotent per `sale_id`.
- **Referral tree visualization:** subtree query from closure table → render an interactive tree (depth-limited, lazy-expand). Analytics: team size, conversion %, revenue attributed, growth over time.
- **Fraud detection:** self-referral block, same-device/IP clustering, signup velocity caps, disposable email/number heuristics, OTP-gated conversion (already enforced). Flag → `fraud_review` queue; don't silently drop.

- Camera/upload capture; pull GPS + timestamp + device metadata (EXIF subset) client-side, re-verify server-side.
- Auto-compose ad: overlay user name/mobile/QR/referral link + company branding + property location/GPS/date/time + badges ("Real Property", "Captured Live", "Taken On Location", "Available Now") + category/price/area/contact.
- Emit all formats: social post, flyer, story, status, reel cover, landscape banner, WhatsApp/Telegram/KaroChat image. Each format = a preset canvas size + slot layout reused from Module B's compositor.

Module E — Field Agent Discovery (Branch 12)

- Agent role captures land/building/construction photos, site-visit media, property videos, voice notes; adds property + owner + price details; submits.
- Lead workflow: `submitted → verifying → approved → published` (or `rejected`). On publish, create the marketplace listing and link `listing_id`.
- Encrypt owner PII at rest; restrict via RLS to the submitting agent + reviewers.

Module F — Viral Marketing System (Branch 13)

- Guarantee every shared asset embeds: referral code, QR, tracking param, dynamic landing page, promoter contact, attribution metadata. (This is enforced by Modules B & D — Module F is the cross-cutting verification + analytics layer.)
- Viral-loop instrumentation: shares → views → clicks → registrations → re-shares, with funnel analytics per asset, per promoter, per channel.

Module G — Gamification & Leaderboards (Branch 14)

- Achievements (first referral, first sale, top agent/promoter/state-head, highest revenue, fastest-growing team, most-shared content) awarded by the points/achievement engine off ledger events.
- Rewards: bonus commissions, badges, recognition, cash rewards, trips, incentives. Leaderboards from materialized views, refreshed on schedule + on demand.

Module H — AI Content Generation (Branch 15)

- **Text (Claude API / NEXUS):** flyers copy, property ads, social posts, WhatsApp/Telegram messages, email campaigns, property descriptions, video scripts, voiceover scripts. Structured JSON outputs where the UI maps them to fields.
- **Image enhancement (adapter pattern):** improve photos, remove backgrounds, add branding/contact/QR/watermark, produce creatives. Define an `ImageEnhancer`

interface with a default provider + room to swap. Never hardcode one vendor in business logic.

- All AI calls server-side; rate-limit per user; log token/credit usage.

6. SECURITY, RLS & ROLES

- Roles (map to existing RBAC): prospect/registered-promoter, agent, state-head, admin, service.
- RLS: a user reads their own profile, their **subtree** (via closure where `ancestor_id = auth.uid()`), their own renders/captures/commissions. State-heads read their scoped subtree. Admin reads all. Engines use service role.
- Hash IPs, encrypt owner PII, never expose raw LP/internal data to client payloads.
- All ledgers append-only; corrections via reversal rows; full audit trail.

7. 🏛️ COMPLIANCE FLAG (read before building commission logic)

This design has **multi-level commission** characteristics. In India these intersect with the **Consumer Protection (Direct Selling) Rules, 2021** and the **Prize Chits and Money Circulation Schemes (Banning) Act, 1978**, which prohibit pyramid and money-circulation schemes. The safe-harbour line is roughly: **payouts must be anchored to genuine real-estate sales, not to enrolment fees or recruitment itself**. Concretely, build with these guardrails so the structure stays on the right side:

- Commissions trigger on **confirmed property sale events**, never on a new user merely signing up.
- No joining fee / no pay-to-enrol; "unlimited depth" should be **payout-capped** by `commission_plan.max_levels` (a configurable, finite number) even if the *tree* is unlimited.
- Keep recruitment rewards (Branch 13/14) as recognition/non-cash or sale-linked, not standalone "recruit-to-earn" cash.

I'm not a lawyer — please have **Senthil (Legal)** sign off on the `commission_plan` model, the depth cap, and the reward structure before go-live. I've built the schema so these are config values you can constrain, not hardcoded assumptions.

8. BUILD ORDER (phased)

1. **Migrations + RLS** (Section 4) and `viral-shared` types/zod.
2. **referral-engine**: codes/QR/share-slug, closure inserts, attribution resolver, click logging.
3. **Registration integration** (Module A) + `/r/<slug>` handler (Module C click path).
4. **personalization-engine** + Brochure/Asset Studio (Module B).
5. **content-capture** + Capture & Ad Studio (Module D).
6. **Commission distribution** on sale events (Module C) + Referral Dashboard (tree + analytics).
7. **field-discovery** + moderation console (Module E).
8. **gamification-engine** + leaderboards (Module G).
9. **ai-content** text + image-enhancement adapter (Module H).
10. **Viral analytics + fraud review** (Module F) and admin config screens.

Ship each phase behind a feature flag; each must build, typecheck, pass its tests, and not regress Branches 1-7.

9. DEFINITION OF DONE / ACCEPTANCE CRITERIA

- A new user registering via a shared link/QR lands in the sharer's tree **atomically** (closure rows correct at every depth).
 - A user can pick any brochure, get it auto-personalized with their identity + QR + trackable link, and share to all 10 channels including KaroChat.
 - Photo capture produces all 9 ad formats with live-location badges and personal branding.
 - A confirmed sale produces correct, immutable, level-capped commission rows (Decimal-exact, idempotent).
 - Tree visualization, leaderboard, and funnel analytics render from real data.
 - Field-agent leads flow submitted→published into the marketplace.
 - AI generates on-brand copy and enhanced creatives server-side.
 - RLS verified: no user can read outside their subtree; ledgers are append-only; PII encrypted.
 - All money is Decimal/`numeric`; zero floats in currency paths.
-

10. OUTPUT EXPECTATIONS FOR THE CODING SESSION

- Start by confirming/adjusting the Section 1 assumptions, then proceed without further questions unless genuinely blocked.
- Generate real migrations, package code, route handlers/edge functions, and UI — not stubs.
- Use Decimal.js, Zod, RLS, and idempotency keys everywhere specified.
- Flag the two open decisions before finalizing commission code:
 1. **Attribution default** — confirm `first_click` (recommended) vs `last_click`.
 2. **Commission depth cap & level rates** — confirm `max_levels` and `level_rates` with Legal before wiring payouts.